

SQL Part 6: Window Functions Made Easy (ROW_NUMBER, RANK, LAG, LEAD) — Free Practice Guide

Beyond School · Ready to learn in a fun way! Anyone can learn — easy! Nothing to install, just a web browser.

Step 1: Open a free online SQL playground

- sqliteonline.com — choose "SQLite", paste, click Run.
- db-fiddle.com — pick "SQLite" as the database, paste and Run.
- sqlime.org — a simple SQLite editor in your browser.
- sqlbolt.com — free interactive lessons for extra practice.

Step 2: Paste this setup script and click Run

It creates the same library tables used in the video. If anything goes wrong, just run it again in a fresh playground.

```
CREATE TABLE books (
  id INTEGER PRIMARY KEY,
  title TEXT,
  author TEXT,
  year INTEGER,
  genre TEXT,
  pages INTEGER
);
INSERT INTO books VALUES
  (1, 'Harry Potter', 'J.K. Rowling', 1997, 'Fantasy', 309),
  (2, 'Matilda', 'Roald Dahl', 1988, 'Comedy', 240),
  (3, 'Wonder', 'R.J. Palacio', 2012, 'Drama', 310),
  (4, 'The Hobbit', 'J.R.R. Tolkien', 1937, 'Fantasy', 310),
  (5, 'Holes', 'Louis Sachar', 1998, 'Mystery', 233),
  (6, 'Charlotte''s Web', 'E.B. White', 1952, 'Classic', 192),
  (7, 'Dog Man', 'Dav Pilkey', 2016, 'Comedy', 240),
  (8, 'Coraline', 'Neil Gaiman', 2002, 'Fantasy', 162);

CREATE TABLE members (
  member_id INTEGER PRIMARY KEY,
  name TEXT,
```

```
  city TEXT,
  joined TEXT,
  email TEXT
);
INSERT INTO members VALUES
  (1, 'Aisha', 'Austin', '2023-01-15', 'aisha@mail.com'),
  (2, 'Ben', 'Denver', '2023-03-02', NULL),
  (3, 'Carlos', 'Austin', '2024-06-20', 'carlos@mail.com'),
  (4, 'Dana', 'Boston', '2024-09-10', NULL),
  (5, 'Eli', 'Denver', '2025-02-01', 'eli@mail.com');

CREATE TABLE loans (
  loan_id INTEGER PRIMARY KEY,
  book_id INTEGER,
  member_id INTEGER,
  loan_date TEXT,
  return_date TEXT
);
INSERT INTO loans VALUES
  (1, 1, 1, '2026-01-05', '2026-01-19'),
  (2, 3, 1, '2026-02-10', NULL),
  (3, 2, 2, '2026-01-20', '2026-02-03'),
  (4, 4, 3, '2026-03-01', '2026-03-15'),
  (5, 1, 3, '2026-04-02', NULL),
  (6, 7, 4, '2026-04-10', '2026-04-24'),
  (7, 7, 2, '2026-05-01', NULL),
  (8, 5, 4, '2026-05-15', NULL);
```

Step 3: Run every query from the video, in order

Paste one at a time, click Run, and compare with the expected result.

AVG OVER: every row stays

```
SELECT title, pages,        AVG(pages) OVER () AS avg_pages FROM books;
```

```
Expected result:
title | pages | avg_pages
Harry Potter | 309 | 249.5
Matilda | 240 | 249.5
Wonder | 310 | 249.5
The Hobbit | 310 | 249.5
Holes | 233 | 249.5
Charlotte's Web | 192 | 249.5
Dog Man | 240 | 249.5
Coraline | 162 | 249.5
```

ROW_NUMBER

```
SELECT ROW_NUMBER() OVER (ORDER BY pages DESC) AS num,        title, pages FROM books;
```

```
Expected result:
num | title | pages
1 | Wonder | 310
2 | The Hobbit | 310
3 | Harry Potter | 309
4 | Matilda | 240
5 | Dog Man | 240
6 | Holes | 233
7 | Charlotte's Web | 192
8 | Coraline | 162
```

RANK and DENSE_RANK

```
SELECT title, pages,    RANK() OVER (ORDER BY pages DESC) AS rnk,    DENSE_RANK() OVER (ORDER BY
```

```
pages DESC) AS dense FROM books;
```

```
Expected result:
title | pages | rnk | dense
Wonder | 310 | 1 | 1
The Hobbit | 310 | 1 | 1
Harry Potter | 309 | 3 | 2
Matilda | 240 | 4 | 3
Dog Man | 240 | 4 | 3
Holes | 233 | 6 | 4
Charlotte's Web | 192 | 7 | 5
Coraline | 162 | 8 | 6
```

Rank inside each genre

```
SELECT genre, title, pages,   RANK() OVER (PARTITION BY genre                ORDER BY pages DESC) AS rnk FROM books;
```

```
Expected result:
genre | title | pages | rnk
Classic | Charlotte's Web | 192 | 1
Comedy | Matilda | 240 | 1
Comedy | Dog Man | 240 | 1
Drama | Wonder | 310 | 1
Fantasy | The Hobbit | 310 | 1
Fantasy | Harry Potter | 309 | 2
Fantasy | Coraline | 162 | 3
Mystery | Holes | 233 | 1
```

Top 1 per group

```
SELECT genre, title, pages FROM (   SELECT genre, title, pages,     ROW_NUMBER() OVER (PARTITION BY genre       ORDER BY pages DESC, year) AS rn   FROM books) WHERE rn = 1;
```

```
Expected result:
genre | title | pages
Classic | Charlotte's Web | 192
Comedy | Matilda | 240
Drama | Wonder | 310
Fantasy | The Hobbit | 310
Mystery | Holes | 233
```

SUM OVER: running total

```
SELECT loan_date, loan_id,    COUNT(*) OVER (ORDER BY loan_date) AS so_far FROM loans ORDER BY loan_date;
```

```
Expected result:
loan_date | loan_id | so_far
2026-01-05 | 1 | 1
2026-01-20 | 3 | 2
2026-02-10 | 2 | 3
2026-03-01 | 4 | 4
2026-04-02 | 5 | 5
2026-04-10 | 6 | 6
2026-05-01 | 7 | 7
2026-05-15 | 8 | 8
```

Running total of pages

```
SELECT l.loan_date, b.title, b.pages,    SUM(b.pages) OVER (ORDER BY l.loan_date) AS total FROM loans AS l JOIN books AS b ON l.book_id = b.id WHERE l.member_id = 3;
```

```
Expected result:
loan_date | title | pages | total
2026-03-01 | The Hobbit | 310 | 310
2026-04-02 | Harry Potter | 309 | 619
```

Running total per member

```
SELECT m.name, l.loan_date,    COUNT(*) OVER (PARTITION BY m.name      ORDER BY l.loan_date) AS so_far FROM loans AS l JOIN members AS m ON l.member_id = m.member_id;
```

```
Expected result:
name | loan_date | so_far
Aisha | 2026-01-05 | 1
Aisha | 2026-02-10 | 2
Ben | 2026-01-20 | 1
Ben | 2026-05-01 | 2
Carlos | 2026-03-01 | 1
Carlos | 2026-04-02 | 2
Dana | 2026-04-10 | 1
Dana | 2026-05-15 | 2
```

LAG: compare with the previous row

```
SELECT loan_id, loan_date,   LAG(loan_date) OVER (ORDER BY loan_date) AS previous FROM loans ORDER BY loan_date;
```

```
Expected result:
loan_id | loan_date | previous
1 | 2026-01-05 | NULL
3 | 2026-01-20 | 2026-01-05
2 | 2026-02-10 | 2026-01-20
4 | 2026-03-01 | 2026-02-10
5 | 2026-04-02 | 2026-03-01
6 | 2026-04-10 | 2026-04-02
7 | 2026-05-01 | 2026-04-10
8 | 2026-05-15 | 2026-05-01
```

LEAD: peek at the next row

```
SELECT title, pages,   LEAD(title) OVER (ORDER BY pages) AS next_longer FROM books ORDER BY pages;
```

```
Expected result:
title | pages | next_longer
Coraline | 162 | Charlotte's Web
Charlotte's Web | 192 | Holes
Holes | 233 | Matilda
Matilda | 240 | Dog Man
Dog Man | 240 | Harry Potter
Harry Potter | 309 | Wonder
Wonder | 310 | The Hobbit
The Hobbit | 310 | NULL
```

LAG with math

```
SELECT title, pages,    pages - LAG(pages) OVER (ORDER BY pages) AS extra FROM books ORDER BY pages;
```

```
Expected result:
title | pages | extra
Coraline | 162 | NULL
Charlotte's Web | 192 | 30
Holes | 233 | 41
Matilda | 240 | 7
Dog Man | 240 | 0
Harry Potter | 309 | 69
Wonder | 310 | 1
The Hobbit | 310 | 0
```

Step 4: Challenges (try first!)

C1. Rank members by how many loans they have made.

C2. Number each member's loans 1, 2, 3 in date order.

C3. Show each book's year and the year of the next newer book.

Answers

C1. Rank members by how many loans they have made.

```
SELECT m.name, COUNT(*) AS loans,
  RANK() OVER (ORDER BY COUNT(*) DESC) AS rnk
```

```
FROM loans AS l JOIN members AS m ON l.member_id = m.member_id
GROUP BY m.name;
```

```
Expected result:
name | loans | rnk
Dana | 2 | 1
Carlos | 2 | 1
Ben | 2 | 1
Aisha | 2 | 1
```

C2. Number each member's loans 1, 2, 3 in date order.

```
SELECT member_id, loan_date,
  ROW_NUMBER() OVER (PARTITION BY member_id ORDER BY loan_date) AS nth
FROM loans;
```

```
Expected result:
member_id | loan_date | nth
1 | 2026-01-05 | 1
1 | 2026-02-10 | 2
2 | 2026-01-20 | 1
2 | 2026-05-01 | 2
3 | 2026-03-01 | 1
3 | 2026-04-02 | 2
4 | 2026-04-10 | 1
4 | 2026-05-15 | 2
```

C3. Show each book's year and the year of the next newer book.

```
SELECT title, year, LEAD(year) OVER (ORDER BY year) AS next_year
FROM books;
```

```
Expected result:
title | year | next_year
The Hobbit | 1937 | 1952
Charlotte's Web | 1952 | 1988
Matilda | 1988 | 1997
Harry Potter | 1997 | 1998
Holes | 1998 | 2002
Coraline | 2002 | 2012
Wonder | 2012 | 2016
Dog Man | 2016 | NULL
```

Keep going!

Watch the whole free SQL course on Beyond School, starting with SQL Part 1: https://youtu.be/72JvYu2OtiA

Beyond School — Ready to learn in a fun way!